asio多线程模型IOServicePool

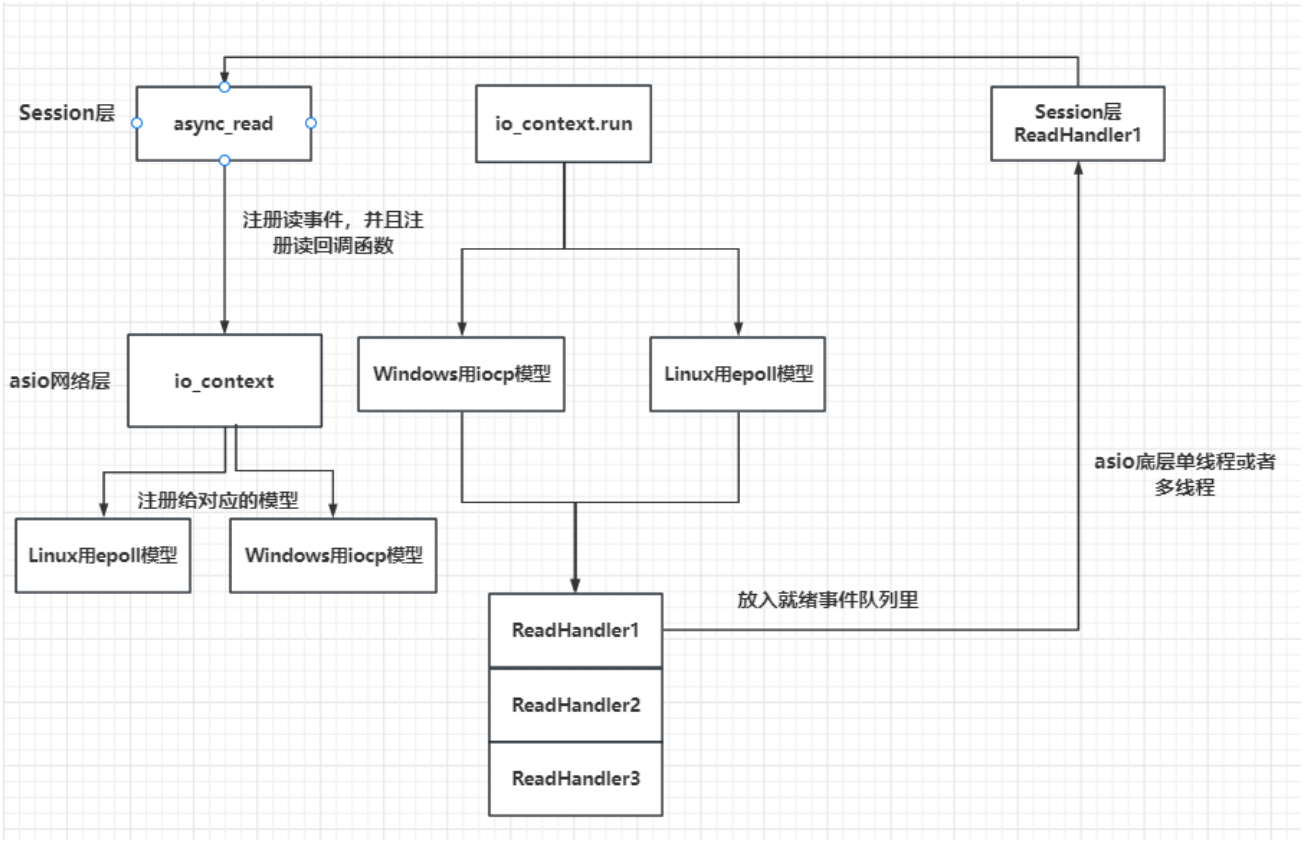
Posted on 2023-06-05 10:24:56 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2Qld2hoFIu8ycYBJXQdxwyWEBfy)

简介

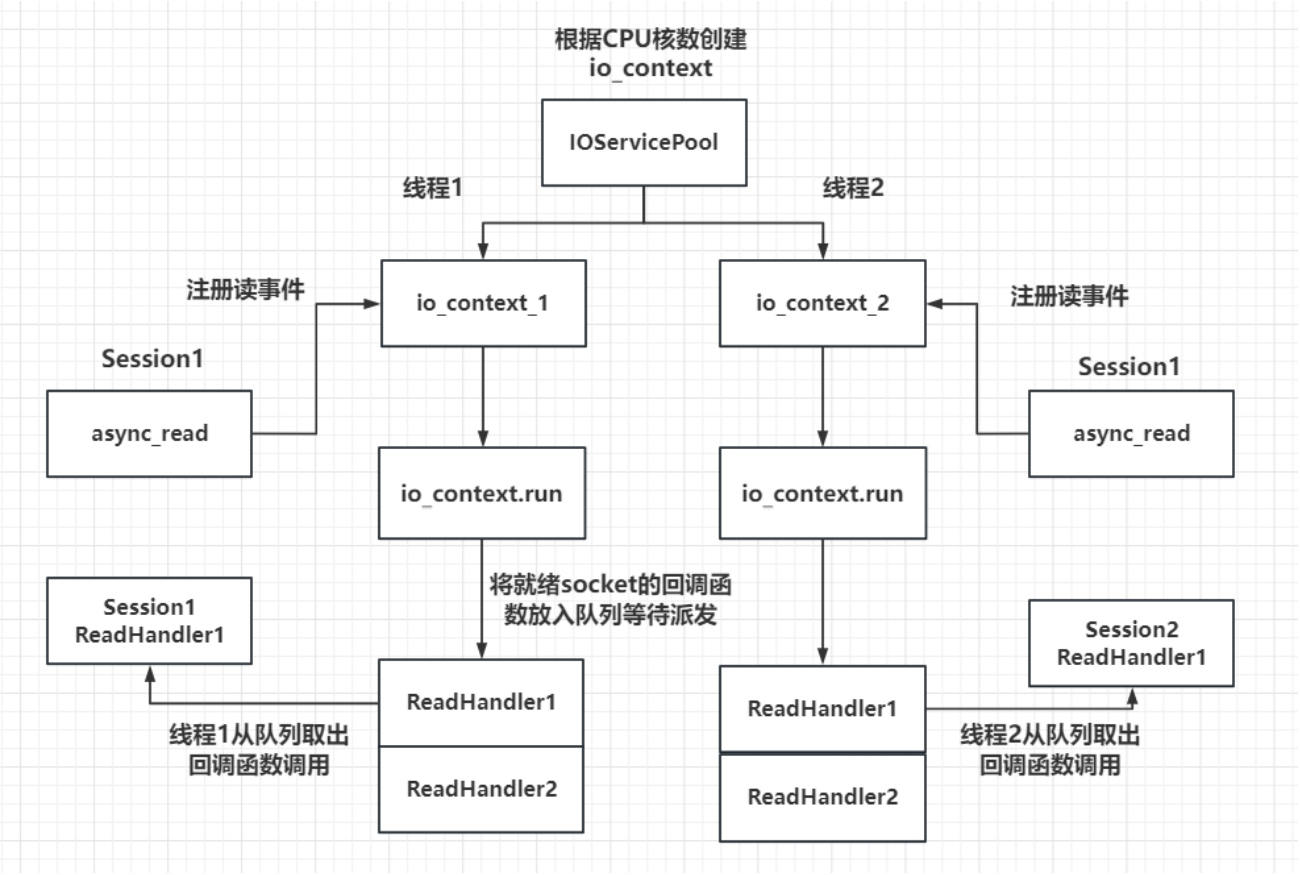
前面的设计，我们对asio的使用都是单线程模式，为了提升网络io并发处理的效率，这一次我们设计多线程模式下asio的使用方式。总体来说asio有两个多线程模型，第一个是启动多个线程，每个线程管理一个iocontext。第二种是只启动一个iocontext，被多个线程共享，后面的文章会对比两个模式的区别，这里先介绍第一种模式，多个线程，每个线程管理独立的iocontext服务。

单线程和多线程对比

之前的单线程模式图如下



我们设计的IOServicePool类型的多线程模型如下：



IOServicePool多线程模式特点

1 每一个io_context跑在不同的线程里，所以同一个socket会被注册在同一个io_context里，它的回调函数也会被单独的一个线程回调，那么对于同一个socket，他的回调函数每次触发都是在同一个线程里，就不会有线程安全问题，网络io层面上的并发是线程安全的。

2 但是对于不同的socket，回调函数的触发可能是同一个线程(两个socket被分配到同一个io_context)，也可能不是同一个线程(两个socket被分配到不同的io_context里)。所以如果两个socket对应的上层逻辑处理，如果有交互或者访问共享区，会存在线程安全问题。比如socket1代表玩家1，socket2代表玩家2，玩家1和玩家2在逻辑层存在交互，比如两个玩家都在做工会任务，他们属于同一个工会，工会积分的增加就是共享区的数据，需要保证线程安全。可以通过加锁或者逻辑队列的方式解决安全问题，我们目前采取了后者。

3 多线程相比单线程，极大的提高了并发能力，因为单线程仅有一个io_context服务用来监听读写事件，就绪后回调函数在一个线程里串行调用, 如果一个回调函数的调用时间较长肯定会影响后续的函数调用，毕竟是穿行调用。而采用多线程方式，可以在一定程度上减少前一个逻辑调用影响下一个调用的情况，比如两个socket被部署到不同的iocontext上，但是当两个socket部署到同一个iocontext上时仍然存在调用时间影响的问题。不过我们已经通过逻

辑队列的方式将网络线程和逻辑线程解耦合了，不会出现前一个调用时间影响下一个回调触发的问题。

IOServicePool实现

IOServicePool本质上是一个线程池，基本功能就是根据构造函数传入的数量创建n个线程和iocontext，然后每个线程跑一个iocontext，这样就可以并发处理不同iocontext读写事件了。

IOServicePool的声明

```
1. class AsioIOServicePool:public Singleton<AsioIOServicePool>
2. {
3.     friend Singleton<AsioIOServicePool>;
4. public:
5.     using IOService = boost::asio::io_context;
6.     using Work = boost::asio::io_context::work;
7.     using WorkPtr = std::unique_ptr<Work>;
8.     ~AsioIOServicePool();
9.     AsioIOServicePool(const AsioIOServicePool&) = delete;
10.    AsioIOServicePool& operator=(const AsioIOServicePool&) = delete;
11.    // 使用 round-robin 的方式返回一个 io_service
12.    boost::asio::io_context& GetIOService();
13.    void Stop();
14. private:
15.    AsioIOServicePool(std::size_t size = std::thread::hardware_concurrency());
16.    std::vector<IOService> _ioServices;
17.    std::vector<WorkPtr> _works;
18.    std::vector<std::thread> _threads;
19.    std::size_t  _nextIOService;
20. };
```

1 `_ioServices` 是一个IOService的vector变量，用来存储初始化的多个IOService。

2 `WorkPtr` 是 `boost::asio::io_context::work` 类型的unique指针。
在实际使用中，我们通常会将一些异步操作提交给 `io_context` 进行处理，然后该操作会被异步执行，而不会立即返回结果。如果没有其他任务需要执行，那么 `io_context` 就会停止工作，导致所有正在进行的异步操作都被取消。这时，我们需要使用 `boost::asio::io_context::work` 对象来防止 `io_context` 停止工作。

`boost::asio::io_context::work` 的作用是持有一个指向 `io_context` 的引用，并通过创建一

个“工作”项来保证 `io_context` 不会停止工作，直到work对象被销毁或者调用 `reset()` 方法为止。当所有异步操作完成后，程序可以使用 `work.reset()` 方法来释放 `io_context` ，从而让其正常退出。

3 `_threads` 是一个线程vector,管理我们开辟的所有线程。

4 `_nextIOService` 是一个轮询索引，我们用最简单的轮询算法为每个新创建的连接分配io_context.

5 因为IOServicePool不允许被copy构造，所以我们将其拷贝构造和拷贝复制函数置为delete

接下来我们实现构造函数

```
1. AsioIOServicePool::AsioIOServicePool(std::size_t size):_ioServices(size),
2. _works(size), _nextIOService(0){
3.     for (std::size_t i = 0; i < size; ++i) {
4.         _works[i] = std::unique_ptr<Work>(new Work(_ioServices[i]));
5.     }
6. 
7.     //遍历多个ioservice，创建多个线程，每个线程内部启动ioservice
8.     for (std::size_t i = 0; i < _ioServices.size(); ++i) {
9.         _threads.emplace_back([this, i]() {
10.             _ioServices[i].run();
11.             });
12.     }
13. }
```

_works是unique_ptr的vector类型，所以初始化时要么放在构造函数初始化列表里初始化，要么通过一个临时的 `std::unique_ptr` 右值初始化，我们采取的是第二种。

实现获取 `io_context&` 的函数

```
1. boost::asio::io_context& AsioIOServicePool::GetIOService() {
2.     auto& service = _ioServices[_nextIOService++];
3.     if (_nextIOService == _ioServices.size()) {
4.         _nextIOService = 0;
5.     }
6.     return service;
7. }
```

我们根据 `_nextIOService` 作为索引，轮询获取 `io_context&` 。

同样我们要实现Stop函数，控制 `AsioIOServicePool` 停止的行为。因为我们要保证每个线程安全退出后再让 `AsioIOServicePool` 停止。

```
1. void AsioIOServicePool::Stop(){
2.     for (auto& work : _works) {
3.         work.reset();
4.     }
5. 
6.     for (auto& t : _threads) {
7.         t.join();
8.     }
9. }
```

其中 `work.reset()` 是让unique指针置空并释放，那么work的析构函数就会被调用，work被析构，其管理的io_service在没有事件监听时就会被释放。

优雅退出

IOServicePool多线程服务器退出时，需要捕获退出信号如SIGINT,SIGTERM等，将退出信号和一个iocontext绑定，当收到退出信号时，我们将IOServicePool停止，并且停止iocontext即可。

```
int main()
{
    try {
        auto pool = AsioIOServicePool::GetInstance();
        boost::asio::io_context  io_context;
        boost::asio::signal_set signals(io_context, SIGINT, SIGTERM);
        signals.async_wait([&io_context,pool](auto, auto) {
            io_context.stop();
            pool->Stop();
            });
        CServer s(io_context, 10086);
        io_context.run();
    }
    catch (std::exception& e) {
        std::cerr << "Exception: " << e.what() << endl;
    }

}
```

上例中，我们将一个iocontext绑定给Server，当收到退出消息后停止这个iocontext，并且停止IOServicePool。

总结

本文总结了如何使用IOServicePool模式构造多线程模型的asio服务器。

视频连接https://space.bilibili.com/271469206/channel/collectiondetail?sid=313101 (https://space.bilibili.com/271469206/channel/collectiondetail?sid=313101)

源码链接https://gitee.com/secondtonone1/boostasio-learn (https://gitee.com/secondtonone1/boostasio-learn)

浏览(3691) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(505) (JavaScript:void(0);)

热门评论

H B I

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



恋恋风辰

北京 朝阳



扫一扫上面的二维码图案，加我为朋友。

个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。

京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)

京ICP备2021033081号 (https://beian.miit.gov.cn/)